阿里云 Data+AI 技术服务专家（公共云方向）主管面试题库

一、项目深挖：AI4Test 多智能体系统

1. 请介绍一下你的 AI4Test 项目

这个项目主要面向 5G NR 基带软件测试场景，希望通过 Agent 系统将需求理解、测试设计、用例生成、环境调度、测试执行、故障分析等环节串联起来。

传统测试流程存在几个问题：第一，测试链路比较长，上下游依赖较多；第二，需求、测试规范、配置、执行日志等数据分散在不同载体中；第三，测试失败后需要人工收集证据、分析原因，定位成本比较高；第四，固定流程难以覆盖不同类型的测试任务和异常情况。

针对这些问题，我参与建设了自研的多智能体系统，通过 Planner-Executor 组织任务规划与执行，将不同领域的能力拆分为多个 Agent，并通过 Skill 封装可复用的工具能力。

系统整体包含任务规划、测试设计、环境分析、用例构建、执行分析、故障定位和结果输出等环节。同时结合 RAG 检索领域知识，并通过状态管理、异常处理和结果校验提高复杂任务的稳定性。

我认为这个项目的核心不只是接入大模型，而是把模型的推理能力、外部工具、业务知识、执行状态和验证机制整合成一个可以落地的工程系统。

2. 为什么选择多 Agent，而不是单 Agent?

我主要从任务复杂度、能力边界和工程可维护性三个方面考虑。

第一，测试任务涉及需求理解、用例生成、环境分析、执行和故障诊断等不同阶段，各阶段需要的知识、工具和输入输出并不相同。

第二，如果全部交给单个 Agent，随着任务复杂度增加，提示词、上下文和工具集合都会膨胀，容易出现职责混乱、工具选择不稳定和错误难以定位的问题。

第三，将职责拆分后，可以分别定义输入输出、工具权限、执行约束和评估标准，也更容易对单个环节进行测试和优化。

但多 Agent 并不是越多越好。拆分会带来通信成本、上下文传递成本和错误传播风险。如果任务简单、步骤固定，普通函数或 Workflow 可能更高效。

所以我的设计思路是：职责边界清晰、需要独立推理或不同工具能力的环节才考虑拆分；确定性的步骤尽量通过程序和 Workflow 实现。

3. 为什么设计 8 个 Agent 和 16 个 Skill?

这两个数字应该由业务职责和能力复用情况决定，而不是为了体现系统复杂度。

Agent 主要按照相对独立的业务职责和任务阶段拆分。例如，不同阶段可能需要不同的上下文、决策逻辑、工具权限和评估标准，因此适合建立独立的 Agent 边界。

Skill 则更偏向可复用的能力封装，例如某种知识查询、配置检查、测试执行或结果解析能力。一个 Agent 可以组合多个 Skill，同一个 Skill 在权限和输入条件允许时，也可以被多个 Agent 复用。

如果把每个工具都做成 Agent，会增加调度和通信开销；如果把所有能力都塞进一个 Skill，又会导致职责过重、输入输出难以管理。

因此，我会根据职责独立性、能力复用率、工具调用频率、维护成本和评估结果来调整拆分方式。面试时我会结合项目中的实际模块，说明每个 `Agent` 和 `Skill` 的具体职责，而不是只强调数量。

4. Planner-Executor 的工作机制是什么？

`Planner-Executor` 的核心是将任务规划和任务执行解耦。

`Planner` 接收用户目标和当前上下文，将任务拆分为有依赖关系的子任务，并确定执行顺序、所需能力和完成条件。

`Executor` 根据规划结果调用对应的 `Agent`、`Skill` 或工具，执行任务并返回结构化结果。

执行过程中，系统需要检查任务状态、工具调用结果和执行异常。如果某一步失败，可以根据错误类型决定重试、调整参数、重新规划或终止任务。

例如，生成测试用例之前，需要先分析需求和测试规范。如果需求信息不足，就不能直接进入用例生成，而应该补充检索或要求澄清。

这里的关键不是让 `Planner` 一次性生成一个永远正确的计划，而是让规划结果能够被执行、观察和验证，并根据真实执行反馈进行调整。

5. Agent 如何决定调用哪个工具？工具调用如何串行或并行？

我会把工具调用拆成依赖分析、执行规划和运行时控制三个阶段。

首先，根据任务目标和工具的功能描述，判断当前需要完成哪些子任务，并明确每个工具的输入、输出和副作用。

其次，根据依赖关系构建任务 `DAG`。如果两个任务只依赖相同的已有输入、彼此没有数据依赖和资源冲突，就可以考虑并行执行；如果后一个任务需要前一个任务的输出，就必须串行执行。

例如，分析测试失败时，可以并行查询历史缺陷、读取相关日志和检查版本配置；但最终根因判断需要综合这些证据，不能在证据收集之前直接完成。

最后，执行层还需要控制并发数、超时、重试、权限和资源占用。涉及环境修改、测试启动等有副作用的操作，还要考虑互斥、幂等和操作授权。

因此，模型可以参与任务规划，但并行与串行的最终执行约束应该由程序根据依赖关系和安全策略进行控制。

6. 如何保证 Agent 生成的测试用例覆盖全面？

我不会只通过模型自评或人工阅读来判断测试用例是否全面，而是建立多层验证机制。

第一，建立需求到测试点的映射。将需求中的功能点、约束条件、边界条件和异常场景结构化，形成可追踪的测试点清单。

第二，结合测试规范、历史用例和缺陷记录进行检索，补充已有经验和容易遗漏的场景。

第三，对生成结果进行规则校验，例如必填字段、参数范围、前置条件、预期结果和用例之间的重复性检查。

第四，将需求覆盖率、规则校验结果、历史缺陷覆盖情况等结合起来评估。对于关键需求，还需要测试人员评审或实际执行验证。

第五，对漏测和失败案例进行分析，将问题区分为需求理解错误、知识检索遗漏、生成逻辑错误和验证机制不足，再针对性改进。

需要注意，覆盖率达到某个比例并不能证明测试绝对全面。最终还需要结合需求追踪、风险分析、实际执行结果和专家评审。

7. 测试执行失败后，如何定位根因?

我会把失败定位看成一个证据驱动的诊断过程，而不是直接让模型根据错误日志猜原因。

第一步是对失败进行初步分类，例如环境异常、版本不匹配、用例构造错误、测试执行异常或潜在软件缺陷。

第二步是收集证据，包括执行日志、失败节点、环境状态、软件版本、测试配置、历史失败记录和领域知识。

第三步是生成候选原因，并为每个候选原因关联支持证据和反向证据。例如，某条日志可能只是异常传播后的表现，不一定是最早发生问题的位置。

第四步是按风险和验证成本安排检查。能够通过读取配置或查询状态验证的问题，优先进行只读检查；需要修改环境或重新执行测试的操作，则应经过权限和风险控制。

第五步是验证假设。如果证据不足，就继续收集信息或转交人工，而不是强行输出确定性结论。

最后输出根因、证据链、影响范围、处理建议和验证结果，并将确认后的案例沉淀到知识库。

8. 如何保证长流程中断后可以恢复，而不是从头执行?

我会将长流程设计成具有明确状态和恢复边界的任务系统。

每个关键步骤都需要记录任务 `ID`、输入摘要、执行状态、输出引用和错误信息。对于成本高或者耗时长的步骤，可以在完成后保存检查点。

任务恢复时，系统先读取最近的有效状态，确认哪些步骤已经成功、哪些仍在执行、哪些失败，再从符合恢复条件的节点继续。

但仅仅保存步骤状态还不够。假设一个步骤已经调用外部工具并成功修改了环境，但系统在记录成功状态之前崩溃，恢复时就可能重复执行。

因此，有副作用的步骤需要进一步考虑幂等键、操作记录、执行前状态检查和必要的补偿机制。

恢复机制还要区分可重试错误和不可重试错误。网络超时可能适合有限重试，而参数错误或权限拒绝通常需要修正条件，不能无限重试。

9. 如何评估多 Agent 系统的效果?

我会从任务效果、过程质量、工程稳定性和业务收益几个维度评估。

第一是任务效果，包括端到端任务成功率、需求覆盖情况、测试用例有效率、故障诊断准确性等。

第二是过程质量，包括工具调用成功率、检索召回质量、无效调用比例、任务规划合理性和人工介入率。

第三是工程稳定性，包括平均执行时间、`P95` 延迟、超时率、重试成功率、任务恢复成功率和资源消耗。

第四是业务收益，例如测试自动化覆盖情况、故障分析时间和人工处理成本的变化。

评估时需要明确统计口径和对照基线，尽量使用固定测试集、历史任务和真实业务任务进行验证。

如果某项指标只是预期目标或阶段性结果，我不会将它描述成已经得到生产数据验证的收益。

10. 你在 AI4Test 项目中遇到的最大挑战是什么?

我认为比较大的挑战是，如何让 `Agent` 不仅能够生成看起来合理的分析结果，还能够真正完成复杂测试任务，并在失败时给出可信的定位结论。

一开始需要解决的并不是单纯的模型调用，而是如何理解测试业务、拆分任务、组织领域知识和工具，并建立可验证的执行闭环。

我主要从三个方面推进。

第一，与熟悉测试业务的同事协作，梳理真实测试流程、关键输入输出和常见异常，避免仅从技术角度设计系统。

第二，将复杂任务拆分为职责明确的环节，为工具定义输入输出和执行约束，降低单个 `Agent` 的职责复杂度。

第三，为关键步骤增加结果校验、异常分类和状态管理。对于失败任务，优先收集证据并验证原因，而不是让模型直接给出结论。

这个过程让我认识到，`Agent` 工程化的难点在于把模型的不确定性控制在可观测、可验证和可恢复的系统边界内。

二、项目深挖：RAG 知识库系统

11. 文档中有大量图片、表格和流程图，RAG 系统如何处理?

我会将文档解析、内容理解、索引构建和检索召回分开设计。

第一步是解析文档。对于 `DOCX`、`PDF`、`PPTX` 等格式，提取正文、标题层级、表格、图片及其所在页面或章节信息。

第二步是处理非纯文本内容。对于流程图和架构图，可以结合 `OCR`、视觉模型或图表解析能力提取文本和结构关系；对于复杂表格，需要保留表头、行列关系和单位，避免转换成普通文本后丢失语义。

第三步是建立关联。为提取出的文本、表格和图片说明保存文档 `ID`、章节、页码、版本等元数据，让检索结果能够追溯到原始资料。

第四步是切分和索引。根据文档结构和内容类型选择合适的切分策略，不应把一个完整表格或关键流程随意切断。

第五步是检索与验证。除了向量检索，也可以结合关键词检索；对涉及表格和图片的问题，需要验证召回内容是否完整、是否包含必要上下文。

对于无法可靠解析的图片或扫描件，我会明确标记解析状态，并在必要时保留原始页面作为人工复核依据，而不是假设所有图片都已经被正确理解。

12. 为什么不能只使用向量检索?

因为企业知识库中的查询类型并不一致，不同检索方法有不同优势。

向量检索适合语义表达不同但含义相近的问题；关键词检索适合错误码、版本号、需求编号等精确匹配；`SQL` 适合结构化数据过滤和统计；图检索适合存在明确实体关系和关联路径的知识。

例如，用户询问某个具体故障编号时，关键词检索通常更直接；询问某类故障的处理经验时，向量检索可能更有效；询问某个版本下有多少失败案例，则可能需要查询结构化数据。

因此，我会根据查询意图选择单一检索或组合检索，并通过统一的数据模型和结果格式整合不同来源。

但混合检索也会增加系统复杂度。需要根据评测结果判断哪些路由真正提升了召回率和答案质量，而不是简单地把所有检索组件都接入系统。

13. 如何评估 RAG 的召回和回答质量?

我会把检索质量和生成质量分开评估。

检索层面，重点看 `Recall@K`、`MRR` 或 `NDCG` 等指标，判断相关文档是否被召回，以及相关结果的排序是否合理。

生成层面，关注答案是否有证据支持、是否忠实于检索上下文、是否正确回答问题，以及引用是否能追溯到原始资料。

此外，还需要关注拒答质量、端到端延迟、`Token` 消耗和人工纠错情况。

评测集应该覆盖常见问题、精确编号查询、跨文档问题、信息不足的问题、错误表达和容易混淆的版本信息。

当某个问题回答错误时，我会先判断错误发生在解析、切分、索引、召回、上下文构建还是生成阶段，再决定优化检索策略、文档处理还是模型提示词。

不能把所有 RAG 错误都归因于模型能力不足。

14. 知识库文档更新后，如何保证索引一致性?

我会把文档版本管理、增量处理和索引切换作为一个完整流程。

首先为文档建立稳定 ID、版本号和内容指纹，识别新增、修改和删除的文档。

其次，只重新解析和索引发生变化的内容，并记录每个处理阶段的状态，避免重复处理未发生变化的文档。

对于向量索引、关键词索引和结构化元数据，需要明确它们之间的版本关联，避免部分索引已经更新、其他索引仍保留旧版本。

如果更新涉及多个组件，可以采用新版本索引构建完成后再切换的方式。切换前执行基本校验，切换后保留必要的回滚能力。

同时要处理缓存失效、旧版本清理和并发更新，保证查询不会混用不一致的版本。

具体采用全量重建、增量更新还是双版本切换，需要结合文档规模、更新频率和系统可用性要求确定。

三、大数据技术：基础原理与选型

15. Spark 和 Flink 有什么区别? 什么场景选哪个?

我会从计算模式、延迟要求、状态管理和业务场景几个方面比较。

Spark 在大规模离线计算、SQL 分析和批量 ETL 场景中应用广泛，也支持流处理。其批任务通常按 DAG、Stage 和 Task 组织执行。

Flink 以持续流处理为核心，适合有状态的实时计算、事件时间窗口、实时指标和实时告警等场景，也支持批处理。

例如，每天对历史订单进行汇总，可以优先评估 Spark；如果需要持续处理订单事件，在几秒内更新实时指标，则可以优先评估 Flink。

但不能简单地认为 Spark 只能批处理、Flink 只能流处理。最终还要看业务延迟要求、数据规模、状态规模、容错要求、现有技术栈和运维成本。

16. Spark 为什么会发生 Shuffle? 为什么 Shuffle 可能很慢?

Shuffle 的本质是按照某种分区规则重新分布数据，使后续计算所需的数据能够汇聚到对应分区。

例如，执行 GROUP BY user_id 时，同一个用户的数据可能分布在不同 Executor 上。为了完成最终聚合，系统需要将相关数据重新分区并传输到下游。

Shuffle 可能涉及数据序列化、网络传输、中间数据写入和读取，以及下游合并和计算。

如果数据量很大、网络或磁盘 I/O 成为瓶颈，或者某些分区的数据量明显过大，Shuffle 就可能拖慢整个作业。

我会先通过执行计划和 Spark UI 确认 Shuffle 发生在哪个 Stage，再检查 Shuffle Read/Write、Task 耗时、Spill 和数据分布。

优化时可以考虑减少输入数据、提前过滤、局部聚合、合理分区以及优化 Join 策略。不能只通过增加 Executor 来解决所有 Shuffle 问题。

17. 什么是数据倾斜？如何识别和解决？

数据倾斜是指数据或计算负载分布不均，导致少数 Task 处理远多于其他 Task 的数据，成为整个作业的性能瓶颈。

例如，按用户 ID 聚合时，某个特殊 ID 对应了大量记录，这些记录可能集中到同一个分区，使该 Task 的耗时远高于其他任务。

识别时，我会比较各 Task 的输入记录数、Shuffle Read、执行耗时和资源指标。如果只有少数 Task 明显拖尾，并且数据量也异常大，就进一步检查 Join Key 或聚合 Key 的分布。

解决方法取决于具体场景。小表适合广播时，可以考虑 Broadcast Join；热点聚合可以考虑两阶段聚合或加盐；分区策略不合理时，可以调整分区或评估 AQE。

但增加分区不一定能够解决单个热点 Key 的问题，必须结合数据分布和执行计划验证。

18. Kafka 消息积压如何排查？

我会沿着生产端、Kafka 分区、消费者和下游系统逐层排查。

第一步，确认积压的规模、增长趋势和影响范围，判断是所有分区都积压，还是部分分区出现异常。

第二步，检查生产速率是否突然增长，以及消息大小、生产端重试和写入延迟是否发生变化。

第三步，检查 Consumer Group 的消费进度、消费速率、错误日志、处理耗时和 Rebalance 情况。

第四步，检查分区数量与消费者并行度。如果分区数不足，增加消费者数量不一定能提升该 Consumer Group 的有效并行度。

第五步，检查下游数据库、存储或 API 是否变慢，避免只扩容消费者而进一步加重下游压力。

处理后需要观察消费速率是否持续高于生产速率、积压是否持续下降，以及端到端处理延迟和数据正确性是否恢复。

19. 数据处理任务失败后，如何恢复并避免重复写入？

我会区分计算状态恢复和外部写入的幂等性。

Checkpoint 可以保存计算状态，使支持相关机制的任务在失败后从一致状态恢复，但它并不自动保证任意外部数据库都不会出现重复写入。

例如，任务已经把数据写入数据库，但在记录成功状态之前崩溃，重试时就可能再次写入同一批数据。

因此，计算框架层面需要合理配置状态保存、恢复和重试；写入层面则可以根据业务选择稳定的业务键、Upsert、事务或去重记录。

同时需要明确失败发生在提交前还是提交后，以及 Source、计算引擎和 Sink 是否共同支持所需的一致性语义。

最后通过故障注入、重复执行和恢复测试，验证结果是否完整、一致且没有非预期的重复副作用。

20. 数据处理任务突然从 10 分钟变成 40 分钟，你怎么排查？

我会先确认数据量、SQL、任务配置、上游输入和运行环境是否发生变化，再对比历史执行基线。

第一步查看执行计划和各阶段耗时，确定时间主要消耗在数据扫描、计算、Shuffle 还是结果写入。

第二步检查资源指标，包括 CPU、内存、GC、磁盘、网络和资源等待，判断是否存在资源瓶颈。

第三步检查数据分布。如果大部分 Task 很快完成，少数 Task 明显拖尾，就进一步排查数据倾斜；如果读取量明显增加，则检查分区裁剪、文件数量和存储布局。

第四步根据证据制定优化方案，优先实施影响范围小、可回滚的修改。

最后用具有代表性的数据量对比优化前后的运行时间、资源消耗和结果正确性，并确认是否满足业务 SLA。

21. 数据量持续增长，应该从哪些方面优化?

我会从 SQL、计算资源、存储布局和数据分布四个方面分析。

SQL 层面，检查是否扫描了不必要的数据、是否存在重复计算、Join 是否合理，以及执行计划是否符合预期。

资源层面，检查 CPU、内存、GC、磁盘、网络和并行度，判断瓶颈是计算能力不足还是 I/O 或资源调度问题。

存储层面，检查分区设计、文件格式、压缩方式和小文件问题，减少不必要的读取和元数据开销。

数据分布层面，检查热点 Key、数据倾斜和分区负载不均，必要时调整分区策略或改变计算方式。

优化之前需要建立基线，优化之后还要比较性能、成本和结果正确性。不能简单地认为增加机器就一定能解决问题。

22. Hadoop、YARN、Hive 分别解决什么问题?

我会把它们放在数据平台的整体架构中理解。

HDFS 是 Hadoop 生态中的分布式文件系统，用于存储大规模数据；NameNode 主要管理文件系统元数据，DataNode 负责存储数据块。

YARN 是资源管理和调度框架，ResourceManager 负责集群级资源管理，NodeManager 负责节点上的资源管理与任务容器执行。

Hive 提供面向数据仓库的表结构和 SQL 查询能力，常用于离线数据处理与分析。实际执行可以由不同的计算引擎承担，具体取决于平台配置。

在一个典型的数据处理链路中，数据可能存储在 HDFS 中，通过 Hive 表管理数据，再由 Spark 等计算引擎执行清洗、关联和聚合任务。

需要注意，这些组件属于不同层次，不能把 HDFS、YARN、Hive 和 Spark 当作相同类型的技术。

四、客户问题定位与技术服务能力

23. 客户反馈系统异常，但没有提供完整日志，你怎么处理?

我不会在信息不足时直接猜测根因，而是先明确问题影响，再逐步补齐证据。

第一步，确认发生时间、影响范围、错误表现、复现条件，以及是否影响核心业务或 SLA。

第二步，引导客户提供必要的信息，例如 Request ID、任务 ID、错误码、相关时间段日志、配置变化和近期变更记录。

第三步，优先利用现有监控、日志和调用链定位问题发生在哪一层，判断是客户端、网络、平台服务还是下游依赖。

第四步，针对候选原因补充验证。如果暂时无法确定根因，我会明确说明已确认的事实、尚未确认的假设和下一步排查计划。

第五步，持续向客户同步处理进展。必要时协调产品、研发或云平台团队，并明确问题负责人和下一次反馈时间。

问题恢复后，还需要验证业务功能、总结根因和制定预防措施。

24. 客户说云上任务运行很慢，你如何定位？

我会先判断问题属于计算资源、数据处理逻辑、存储 I/O、网络还是上下游依赖。

首先确认慢的是单个任务还是整个集群，问题从什么时候开始，是否伴随数据量、配置或业务流量变化。

其次检查任务执行计划、阶段耗时和资源监控。如果 CPU 长期饱和，重点看计算逻辑和并行度；如果 Shuffle 或 Spill 明显增加，则检查数据分布、内存和磁盘；如果 CPU 利用率不高但任务很慢，则继续检查 I/O、资源等待和外部依赖。

然后根据证据提出有针对性的优化措施，例如调整 SQL、优化分区、处理倾斜或合理调整资源。

执行变更前，我会评估风险和回滚方案；执行后对比耗时、资源成本和结果正确性，确认优化确实有效。

25. 客户坚持认为是云平台的问题，但目前没有证据，你怎么办？

我会避免直接与客户争论问题归属，而是把沟通重点放在共同确认事实和恢复业务上。

首先确认客户遇到的实际影响，说明我们会协助定位，同时不在证据不足时提前判断责任方。

然后沿调用链逐层检查，收集请求 ID、错误日志、监控指标、资源使用情况和依赖服务状态，逐步缩小问题范围。

如果证据指向云平台服务，我会按流程升级并跟进；如果证据指向客户配置、应用代码或下游系统，也会提供清晰的证据和处理建议。

如果暂时无法确定，我会说明目前的排查结论、剩余不确定性和下一步计划，并持续同步进度。

技术服务的价值不只是判断问题属于谁，更重要的是尽快控制影响、恢复业务，并通过证据建立客户信任。

26. 问题迟迟没有解决，研发团队也没有及时响应，你怎么办？

我会从业务影响、问题推进和风险控制三个方面处理。

首先明确问题等级和业务影响。如果涉及核心业务中断或严重 SLA 风险，就按照升级机制推动处理，而不是被动等待。

其次整理已经确认的事实、复现步骤、日志、排查结果和待验证假设，减少研发团队重新收集信息的成本。

第三，明确问题负责人、协作团队、下一步动作和时间节点。如果暂时无法根治，可以评估安全的临时缓解措施，但必须说明风险和回滚方式。

在整个过程中，我会定期向客户同步已完成事项、当前结论和下一步计划。

问题解决后，再组织复盘，分析为什么没有及时发现、定位或响应，并形成可复用的排障文档或监控改进措施。

27. 你不是传统大数据专家，客户问到你不会的技术怎么办？

我会先明确自己的经验边界，不会为了表现专业而给出未经验证的结论。

遇到不熟悉的问题，我会先确认现象、影响范围和可获得的证据，再利用监控、日志、官方文档和现有工具缩小问题范围。

对于自己能够验证的部分，我会独立完成分析；涉及底层实现、平台内部机制或超出当前权限的问题，我会及时协调相应的产品和研发专家。

同时，我会向客户说明当前已经确认的事实、还需要验证的部分，以及下一步计划。

我认为技术服务专家需要的不只是掌握很多技术名词，更重要的是具备系统化的问题分析能力、正确的风险意识和推动问题闭环的能力。

对于重复出现的问题，我会将根因、处理步骤和验证方法沉淀下来，逐步减少后续问题的定位成本。

28. 如何平衡客户诉求、技术风险和交付时间?

我会先明确客户真正需要解决的业务问题，再判断技术方案的风险、成本和交付周期。

对于低风险、可逆的优化，可以优先尝试快速验证；对于涉及数据删除、生产配置变更、权限提升或可能影响业务连续性的操作，则需要充分评估风险，必要时设置审批、备份和回滚机制。

如果客户要求的时间与实际工作量不匹配，我会拆分交付目标，明确哪些能力可以先交付，哪些需要后续完善，同时说明每个阶段的风险和验收标准。

我不会为了按时交付而忽略数据正确性和安全性，也不会在缺乏依据时承诺无法保证的结果。

最终目标是在业务价值、交付效率和技术风险之间找到可解释、可验证的平衡。

五、Data Agent 与岗位业务理解

29. 如果让你设计一个企业数据开发 Agent，你会怎么设计?

我会先确认用户需要解决的具体问题，是自然语言生成 SQL、数据开发、任务运维、数据质量检查，还是数据分析。

在此基础上设计完整执行链路。

第一步，理解用户意图和业务指标，补充必要的输入条件。

第二步，查询元数据、表结构、数据血缘、指标定义和权限信息，建立可靠的上下文。

第三步，根据任务类型生成 SQL 或数据处理计划，并对执行依赖进行分析。

第四步，在执行之前进行语法检查、权限校验、成本评估和风险控制。高风险操作需要审批，不能完全由模型自行决定。

第五步，在受控环境执行任务，并检查结果是否符合业务规则。

第六步，记录执行过程、错误和结果，支持失败诊断、重试和人工接管。

评估时，我会关注任务成功率、SQL 正确率、结果可信度、人工介入率、执行成本和响应延迟。

我认为 Data Agent 的核心不是简单地用自然语言生成 SQL，而是把数据语义、权限、安全执行、质量校验和异常恢复整合成可靠的工作流。

30. RAG Agent 与 Data Agent 有什么区别?

RAG 主要解决如何从知识库中检索相关信息，并利用检索结果辅助生成回答。

Data Agent 的范围通常更广，可能需要查询数据目录、理解表结构、生成 SQL、调用数据处理平台、检查数据质量，甚至执行任务和分析异常。

两者可以组合。例如，Data Agent 在生成 SQL 之前，可以通过 RAG 检索业务指标定义、历史 SQL 模板和数据治理规范。

但 RAG 返回的内容主要是知识和证据，Data Agent 还需要决定下一步动作，并处理执行权限、运行状态、结果校验和失败恢复。

因此，我会将 RAG 视为 Data Agent 可以使用的一种知识能力，而不是把两者看成完全相同的系统。

31. 如何将你现有的 AI4Test 经验迁移到 Data+AI 场景?

我认为两类场景的业务领域不同，但底层工程问题有较强的共性。

AI4Test 需要理解测试需求、查询领域知识、调用工具、执行复杂流程，并根据执行结果进行验证和故障恢复。

Data Agent 同样需要理解业务目标、查询元数据和业务规则、生成执行计划、调用数据平台工具，并验证执行结果。

可以迁移的是任务规划、工具封装、上下文管理、RAG、权限控制、状态恢复和评估机制。

需要重新学习的是数据平台本身的技术体系，例如数据仓库、数据湖、Spark、Flink、Kafka、数据治理和云上数据产品。

因此，我不会认为做过 AI4Test 就等于已经具备完整的数据平台经验，而是会利用已有的 Agent 工程经验降低新领域的学习成本，再通过具体业务场景补齐数据技术知识。

32. 如何理解云上数据平台的技术服务价值?

我理解云上数据平台的技术服务不只是解决产品使用问题，而是帮助客户把数据基础设施真正转化为业务能力。

一方面，需要保障平台稳定性，包括任务运行、性能优化、故障排查、权限和数据安全。

另一方面，需要结合客户的数据架构和业务目标，提供数据接入、处理、存储、分析和 AI 应用方面的解决方案。

例如，客户不仅需要把数据处理任务运行起来，还可能需要降低处理延迟、控制资源成本、提高数据质量，或者通过 Data Agent 减少人工数据开发工作。

技术服务专家需要理解这些业务目标，再将它们转化为可实施、可验证和可持续运维的技术方案。

因此，我认为这个岗位既需要技术深度，也需要客户沟通、问题推动和交付意识。

六、岗位匹配、求职动机与职业规划

33. 为什么想从华为转到阿里云?

我目前在华为无线部门主要从事 AI4Test Agent 系统的工程化落地，也参与了面向 5G 基带领域的知识库和 RAG 能力建设。

这段经历让我对 Agent 架构、工具调用、知识检索、复杂任务编排和系统可靠性有了比较深入的实践认识。

下一步，我希望把这些能力应用到更广泛的企业 Data+AI 场景中，接触不同客户的数据架构、技术问题和业务需求，进一步提升从问题分析、方案设计到落地验证的能力。

阿里云的公共云场景能够让我接触云上数据平台、AI 应用和客户实际问题。我希望在已有 AI 工程经验的基础上，补齐传统数据平台和云产品方面的能力，逐步成长为兼具系统设计和实际问题解决能力的技术专家。

34. 为什么选择技术服务专家，而不是纯 Agent 研发?

我长期还是希望走技术专家路线，持续积累系统架构、AI 工程化和复杂问题定位方面的能力。

我理解技术服务专家并不只是负责沟通和协调。面对真实客户问题，需要理解底层技术机制、分析日志和监控、设计解决方案，并通过验证确认问题得到解决。

相比只关注单一内部系统，客户场景也能帮助我更全面地理解数据平台、云产品和企业 AI 应用的实际需求。

当然，这个岗位也会涉及客户沟通、项目推进和交付工作。我愿意承担这些职责，但希望在解决客户问题的过程中持续积累技术深度，而不是只做流程协调。

35. 你没有多年传统大数据经验，为什么认为自己适合这个岗位?

我认为需要客观看待自己的优势和不足。

我的优势是已经参与了复杂 Agent 系统和 RAG 平台的工程化建设，具备任务编排、工具集成、日志分析、异常处理、状态恢复和结果验证方面的实践经验。

这些能力能够迁移到 Data Agent 和部分数据平台问题定位场景。

但我也清楚，自己在传统大数据平台长期生产运维、复杂 Spark/Flink 性能优化以及完整数据平台迁移交付方面，还需要补充实际经验。

因此，我不会把 Agent 开发经验等同于传统大数据经验。我会先系统掌握数据处理架构、核心组件和云产品能力，再结合客户场景进行实践。

我希望依靠已有的工程化基础和持续学习能力，逐步补齐数据平台能力，最终能够独立分析并解决更复杂的客户技术问题。

36. 如果客户问题大部分是传统大数据问题，而不是 AI，你能接受吗?

可以。我理解这个岗位面向的是 Data+AI 整体场景，而不是单纯的 Agent 研发岗位。

客户的 AI 应用最终也依赖数据接入、存储、计算、治理和服务等基础能力。如果基础数据链路存在性能、质量或可靠性问题，Agent 本身也很难产生稳定的业务价值。

所以我愿意补齐传统数据平台的技术基础，先把客户问题定位和解决好，再结合具体需求探索 AI 能力可以发挥的作用。

对我来说，关键是能否在实际工作中持续积累底层技术和系统设计能力，而不是只处理某一种固定类型的问题。

37. 如果入职，前 90 天准备怎么开展工作?

我会分三个阶段推进。

第一个阶段是熟悉产品和业务。了解主要云产品、典型客户架构、常见问题、服务流程和升级机制，同时梳理自己在数据平台方面的知识缺口。

第二个阶段是跟随项目和问题处理进行实践。通过参与客户问题分析、日志排查、方案验证和交付过程，建立对实际业务链路和常见故障的认识。

第三个阶段是逐步独立承担问题，并沉淀可复用的方法。对于高频问题，形成排障文档、检查清单或自动化工具；对于适合 AI 的环节，评估是否能够通过 RAG 或 Agent 提高问题定位效率。

具体目标还需要结合团队当前的业务重点、产品体系和岗位考核要求确定。

38. 未来三年有什么职业规划?

我希望长期沿技术专家路线发展。

第一阶段，我会补齐云上数据平台和核心大数据组件的基础能力，熟悉客户业务场景、云产品架构和典型问题处理方法，形成独立排障和交付能力。

第二阶段，希望能够独立负责更复杂的 Data+AI 场景，具备从需求分析、架构设计、性能优化到上线验证的完整能力，并能推动跨团队解决问题。

第三阶段，希望进一步形成自己的技术专长，将数据平台、Agent 工程化和系统可靠性结合起来，成为既理解底层技术，又能解决复杂业务问题的技术专家。

我不会只把目标设定为掌握更多框架，而是希望能够独立解决复杂问题，并持续沉淀可复用的架构和工程方法。

39. 你有什么问题想问主管？

我想了解一下，这个岗位目前最核心的业务目标是什么？入职前三个月，您最希望新人独立承担哪些工作？

另外，我也想了解一下团队 Data Agent 相关工作的实际形态，是偏底层平台和产品研发，还是偏客户场景中的方案设计、实施和问题解决？不同方向在团队中的工作占比大概是什么样的？

最后，对于这个岗位，您认为表现优秀的人与一般水平的人，最主要的差距体现在哪些方面？

七、面试前的最后检查

第一，项目深挖：
能讲清楚 AI4Test 的整体架构、Agent 与 Skill 的职责、Planner-Executor 的运行机制，以及一个真实故障案例。

第二，大数据技术：
能解释 Spark 和 Flink 的区别、Shuffle 的原理、数据倾斜的识别方法，以及 Kafka 消息积压的排查路径。

第三，分布式可靠性：
能讲清 Checkpoint、重试、幂等和事务的区别，知道如何避免重复写入。

第四，客户问题处理：
能从影响范围、证据收集、根因分析、方案实施、结果验证和客户沟通几个方面回答场景题。

第五，岗位匹配：
能解释为什么选择阿里云、为什么做技术服务、自己有哪些优势和不足，以及未来希望形成什么技术专长。

第六，回答原则：
只说自己真正做过的事情；不懂的技术明确经验边界；回答问题时先讲结论，再讲机制、步骤、取舍和验证方法。

建议练习顺序： 第 1、7、16、17、18、19、20、23、24、29、33、35 题。这些问题最容易串联你的项目经验、大数据基础、故障排查能力和岗位匹配度。